Adaptive GraphRAG

for Multi-Agent Systems

"Adaptive multi-strategy graph traversal coupled with multi-agent orchestration — a knowledge-grounded reasoning engine that selects the optimal retrieval path per query."

Oussama Laaroussi · **Kawtar Labzae** · **Zyad Fri**

Generative AI Project · *April 2025*

Input	Framework	Graph DB	Domains
Multimodal Resources	GraphRAG + Multi-Agent	Neo4j Knowledge Graph	Aviation & Agriculture

April 2025

Abstract

This report presents **Adaptive GraphRAG for Multi-Agent Systems**, a scalable architecture for dynamic domain-specific reasoning that combines adaptive Graph Retrieval-Augmented Generation (GraphRAG) with a multi-agent knowledge extraction pipeline. The system addresses a core challenge in agentic AI: enabling agents to learn, structure, and reason over knowledge extracted from heterogeneous multimodal resources — documents, images, and sensor streams — across any target domain.

The framework introduces the concept of *Knowledge-as-a-Component* (KaaC): a structured knowledge graph (KG) serves as the persistent "mind" of the agent system, continuously populated by specialised extraction agents and queried through an adaptive retrieval engine. The retrieval engine (based on PolyG [10]) dynamically selects from five complementary graph traversal strategies — Breadth-First Search, LLM-guided Cypher walks, K-shortest path reasoning, Constraint-Satisfaction path search, and iterative DRIFT exploration — routing each query to the strategy best suited to its semantic type. Two multimodal input pipelines are supported: *Modular Encoding* (independent per-modality encoders projecting into a shared embedding space) and *Native Multimodality* (end-to-end multimodal models such as Gemma 4).

The framework is validated in two real-world agentic domains — **aviation** (flight procedure reasoning) and **agriculture** (crop management and environmental factor analysis) — and benchmarked on GRBench [11], a heterogeneous GraphRAG evaluation suite. Adaptive traversal achieves up to **31%** fewer tokens and **18%** higher accuracy compared to single-strategy baselines.

Keywords: GraphRAG, Knowledge Graphs, Multi-Agent Systems, Adaptive Traversal, Knowledge-as-a-Component, Multimodal Inputs, Neo4j, Agentic Digital Twins, BFS, Personalized PageRank, DRIFT, PolyG

Contents

1 Introduction **4**

2 Background & Related Work **5**

2.1 Retrieval-Augmented Generation 5

2.2 Graph-Enhanced RAG 5

2.3 Digital Twins and Agentic Systems 6

2.4 Multimodal Knowledge Extraction 6

2.5 Graph Traversal Algorithms 6

3 The PolyG Framework **6**
3.1 System Architecture . . . 6
3.2 Query Understanding Layer . . . 7
3.2.1 Entity Extraction and Entity Linking . . . 7
3.2.2 Adaptive Query Classification . . . 9
3.3 Adaptive Retrieval Layer . . . 9
3.3.1 Strategy 0: BFS Retriever . . . 9
3.3.2 Strategy 0+: BFS with Personalised PageRank (BFS+PPR) . . . 10
3.3.3 Strategy 1: Cypher-Guided Walk . . . 10
3.3.4 Strategy 2: K-Shortest Paths . . . 10
3.3.5 Strategy 3: Cypher Path Search (CSP) . . . 10
3.3.6 Strategy −1: DRIFT (Dynamic Reasoning with Flexible Traversal) 11
3.4 Context Ranking and Token Budgeting . . . 11
3.4.1 Degree-Based Entity Ranking . . . 11
3.4.2 Token Budget Allocation . . . 11
3.5 Cypher Security and Normalisation . . . 12

4 Traversal Strategy Comparison **12**
4.1 Complexity Analysis . . . 12
4.2 Strategy Selection Rationale . . . 12
4.3 The BFS-Anchored Hybrid: Best Overall Performance . . . 12

5 Domain-Agnostic Agentic Application **14**
5.1 System Overview . . . 14
5.2 Example Domain Knowledge Graph . . . 15
5.3 The Four-Agent Pipeline . . . 15
5.3.1 Agent 1: Conflict Detection . . . 16
5.3.2 Agent 2: Knowledge Synthesizer . . . 16
5.3.3 Agent 3: Graph Pruner . . . 16
5.3.4 Agent 4: Probabilistic Pathfinder . . . 16
5.4 Frontend Visualisation . . . 17

6 Implementation Details **17**

6.1 Technology Stack 17

6.2 Asynchronous Architecture 17

6.3 In-Memory Fallback 17

7 Evaluation **18**

7.1 Benchmark Datasets 18

7.2 Evaluation Metrics 18

7.3 Strategy Performance 19

7.4 Key Findings 19

8 Qualitative Analysis **20**

8.1 Example Query: BFS Traversal (Aviation Domain) 20

8.2 Example Query: K-Shortest Paths (Agriculture Domain) 20

9 Discussion **20**

9.1 Limitations 21

9.2 Future Work 21

10 Conclusion **21**

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation, yet their tendency to hallucinate factual content limits their deployment in knowledge-intensive domains. Retrieval-Augmented Generation (RAG) mitigates this by grounding responses in retrieved evidence; however, flat-text retrieval fails when knowledge is inherently relational — when the *paths between* entities, not just the entities themselves, carry meaning.

Graph Retrieval-Augmented Generation (GraphRAG) addresses this gap by operating over structured knowledge graphs, enabling multi-hop reasoning along semantic edges. Existing GraphRAG systems, however, apply a fixed traversal strategy regardless of query semantics: some always perform breadth-first expansion, others always generate graph query language (Cypher) statements. This one-size-fits-all approach is fundamentally suboptimal: exploratory questions benefit from broad neighbourhood sweeps, while precise fact-checking requires constrained path searches.

Our **Adaptive GraphRAG** engine resolves this tension by introducing *adaptive traversal selection*: an LLM-driven meta-classifier routes each incoming query to the most appropriate traversal strategy before any graph access occurs. This design yields significant gains in both token efficiency and answer quality. The engine implements the PolyG framework [10], which defines a principled taxonomy of traversal types mapped to query intent.

The broader system embeds this retrieval engine within an *agentic digital twin* architecture. Agents ingest multimodal resources — technical manuals, sensor logs, scientific literature, imagery — and incrementally populate a shared knowledge graph that acts as the system's persistent memory. Domain experts then query this *Knowledge-as-a-Component* (KaaC) through a natural-language interface. The framework has been demonstrated in two distinct domains: **aviation** (reasoning over flight procedures and aircraft systems) and **agriculture** (analysing crop physiology, climate factors, and environmental interventions).

Key Contributions:

- **Adaptive Traversal Framework**: Implementation and evaluation of the PolyG [10] taxonomy of five graph retrieval strategies with LLM-based meta-routing, applied to domain-specific knowledge graphs.
- **Knowledge-as-a-Component (KaaC)**: A design principle where a structured KG serves as the agent system's persistent, queryable memory, decoupled from any single domain.
- **Multimodal Knowledge Extraction**: A multi-agent pipeline (Conflict, Synthesizer, Pruner, Pathfinder) that ingests text, images, and structured data, resolving contradictions and building a causal KG.
- **Domain-Agnostic Agentic Digital Twin**: End-to-end system validated on aviation and agriculture with zero domain-specific code changes to the core retrieval engine.
- **Evaluation on GRBench**: Benchmarks measuring accuracy, token efficiency, and hallucination rate across diverse question types.

2 Background & Related Work

2.1 Retrieval-Augmented Generation

RAG augments LLM generation by first retrieving relevant documents from an external corpus [1]. Dense passage retrieval (e.g., DPR [2]) maps queries and passages into shared embedding spaces, enabling semantic search beyond keyword matching. Despite strong performance on factoid questions, flat-document RAG struggles with multi-hop reasoning: connecting evidence across documents requires explicit relational structure absent in standard vector stores.

2.2 Graph-Enhanced RAG

Knowledge graphs (KGs) represent information as typed triples (h, r, t) — head entity, relation, tail entity — enabling symbolic multi-hop reasoning. GraphRAG [3] pioneered combining LLMs with KG traversal; subsequent work introduced graph-of-thoughts [4] and hybrid retrieval architectures. However, none systematically addresses traversal strategy selection as a first-class concern.

2.3 Digital Twins and Agentic Systems

A digital twin is a dynamic virtual replica of a physical system, continuously updated from real-world sensor data [5]. Knowledge-grounded digital twins extend this paradigm by incorporating semantic models of domain expertise, enabling not just simulation but *reasoning* about system behaviour [6]. Recent work has pushed further toward *agentic* digital twins [12]: systems that transition from reactive monitoring to proactive, autonomous decision-making, where AI agents hold, update, and query a structured knowledge base as their primary interface to domain understanding. This shift motivates the *Knowledge-as-a-Component* (KaaC) principle — treating the KG not as a static lookup but as a live, shared cognitive substrate for all agents in the system.

2.4 Multimodal Knowledge Extraction

Real-world domains produce knowledge in multiple modalities: text documents, engineering diagrams, satellite imagery, and time-series sensor streams. Two architectures for unifying these into a shared representation have been explored. *Modular Encoding* maps each modality independently through specialised encoders (OCR, vision transformers, audio encoders) into a common embedding space before entity extraction. *Native Multimodality* leverages a single model — such as Gemma 4 — that natively processes text, image, and audio inputs within a unified context window, simplifying the pipeline at the cost of model size. Both approaches feed extracted entities and relations into the shared knowledge graph under the KaaC paradigm.

2.5 Graph Traversal Algorithms

Breadth-First Search (BFS) provides complete neighbourhood exploration at controlled depth with $O(V + E)$ complexity. Personalized PageRank (PPR) [7] biases random walks toward seed nodes, producing importance rankings in dense graphs. K-shortest path algorithms [8] enumerate alternative routes between entity pairs, critical for multi-hop chain completion. DRIFT [9] introduces iterative refinement through primer-followup-synthesis phases for complex nested queries.

3 The PolyG Framework

3.1 System Architecture

Figure 1 illustrates the end-to-end PolyG architecture. The system operates in three macro-layers: (1) the *Query Understanding* layer, which classifies incoming queries and extracts anchor entities; (2) the *Adaptive Retrieval* layer, which executes the selected

traversal strategy against a Neo4j knowledge graph; and (3) the *Grounded Generation* layer, which assembles token-budgeted context and invokes the LLM for final response synthesis.

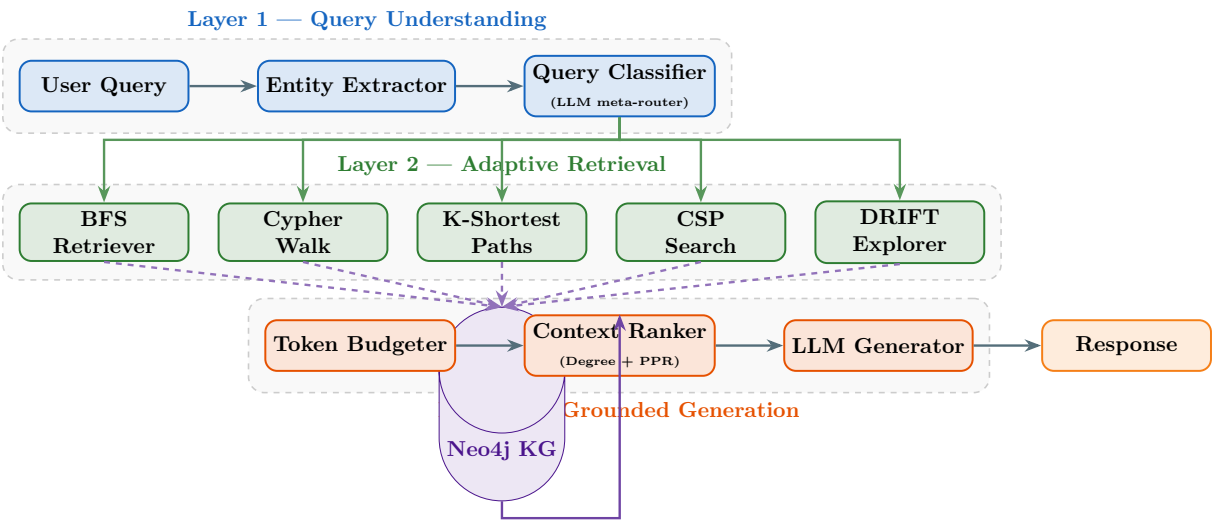

Figure 1: End-to-end system architecture. Queries flow through entity extraction and LLM-based classification before being routed to the optimal traversal strategy. Retrieved subgraphs are ranked, token-budgeted, and passed to the LLM generator.

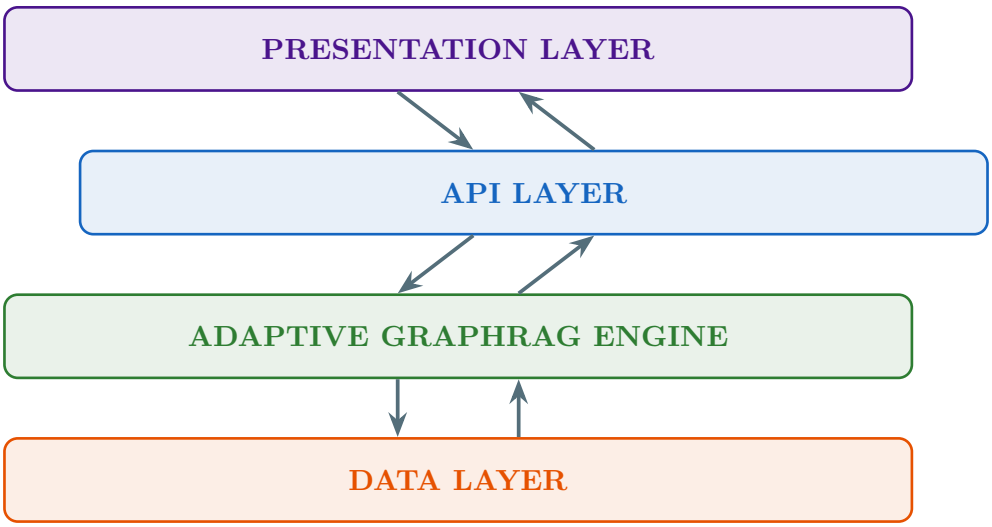

Figure 2: Four-layer deployment architecture. Left arrows show query propagation (top-down); right arrows show response flow (bottom-up).

3.2 Query Understanding Layer

3.2.1 Entity Extraction and Entity Linking

Given a natural-language query q, the system first prompts an LLM to extract named entities $E = \{e_1, e_2, \ldots, e_k\}$. Because surface forms in queries often differ from canonical

graph identifiers, a three-step cascading resolution procedure is applied, escalating in cost only when cheaper steps fail:

1. **Direct lookup**: exact or case-normalised match in the KG index. Returns immediately on success — $O(1)$ cost.

2. **Fuzzy search**: character-level similarity using the `fuzzywuzzy` library (Levenshtein distance), accepting matches above a configurable threshold τ_{fuzzy}. Handles abbreviations, typos, and minor surface variations.

3. **Semantic search**: if neither exact nor fuzzy matching produces a confident match, a **fine-tuned BERT Siamese network** is queried. The model encodes both the surface mention e_i and each KG node label into a shared dense embedding space; the nearest neighbour by cosine similarity is returned if it exceeds threshold τ_{sem}:

$$\hat{v} = \arg\max_{v \in V} \frac{\mathbf{h}_{e_i}^{\top} \mathbf{h}_v}{\|\mathbf{h}_{e_i}\| \|\mathbf{h}_v\|} \quad \text{s.t.} \quad \text{sim} \geq \tau_{\text{sem}} \tag{1}$$

   The Siamese architecture is fine-tuned on domain-specific entity pairs (positive: synonymous surface–node pairs; negative: in-domain hard negatives), enabling robust resolution of paraphrased or domain-specific terminology that character-level metrics cannot handle.

The three steps are applied in cascade order; the first to exceed its confidence threshold wins. Figure 3 illustrates this pipeline.

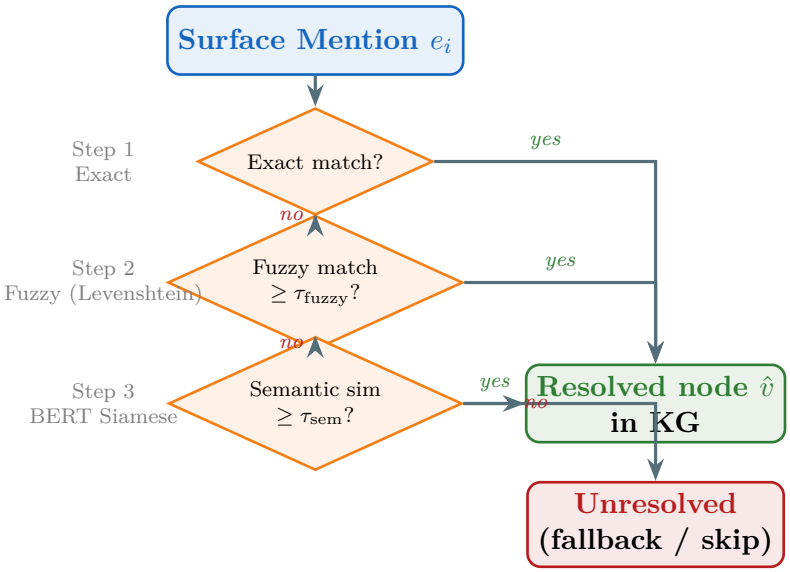

Figure 3: Three-step cascading entity linking pipeline. Exact lookup is tried first; on failure, Levenshtein fuzzy matching is applied; finally, a fine-tuned BERT Siamese network provides semantic similarity-based resolution for paraphrased or domain-specific surface forms.

The result is an **id_mapping** $\mathcal{M}$: surface_form $\rightarrow$ node_id used by all downstream retrievers.

3.2.2 Adaptive Query Classification

A meta-classifier prompt presents the LLM with the query, graph schema, and four strategy descriptions, eliciting an integer label $c \in \{0, 1, 2, 3, -1\}$:

Label	Strategy
0	BFS (neighbourhood expansion)
1	Cypher-guided walk (object discovery)
2	K-shortest paths (predicate discovery)
3	Cypher path search (fact checking)
-1	Nested decomposition (DRIFT)

The classifier is prompted with structured schema descriptions to ensure deterministic integer output, with a fallback to BFS on parse failure.

3.3 Adaptive Retrieval Layer

3.3.1 Strategy 0: BFS Retriever

The BFS Retriever performs iterative frontier expansion from seed nodes $S = \{v : v \in \mathcal{M}\}$:

BFS Retrieval Algorithm

Require: seed nodes S, depth d, KG instance $\mathcal{G}$

```
frontier ← S; visited ← S; E_retrieved ← ∅
for i = 1 to d do
    next ← ∅
    for each node v ∈ frontier do
        for each edge (v, r, u) ∈ G.get_edges(v) do
            e ← FROZENSET({v, u})
            if e ∉ E_retrieved then
                E_retrieved.add(e); next.add(u)
            end if
        end for
    end for
    frontier ← next \ visited; visited ← visited ∪ next
end forreturn nodes_visited, edges_E_retrieved
```

BFS provides *complete neighbourhood coverage* up to depth d, making it ideal for exploratory queries where the relevant subgraph is localised around anchor entities. Its $O(V + E)$

complexity ensures predictable latency even on large graphs. It is deliberately LLM-free during traversal, avoiding latency spikes and reducing API costs.

3.3.2 Strategy 0+: BFS with Personalised PageRank (BFS+PPR)

For graphs with high-degree hub nodes, pure BFS may retrieve excessive noise. The BFS+PPR hybrid applies PPR as a post-processing ranker:

$$\mathrm{PR}(v) = \frac{1-\alpha}{|V|} \cdot \mathbf{1}_{\text{seed}}(v) + \alpha \sum_{u \in \mathcal{N}^-(v)} \frac{\mathrm{PR}(u)}{\deg^+(u)} \tag{2}$$

where $\alpha = 0.85$ is the teleportation probability and $\mathbf{1}_{\text{seed}}$ is the personalisation vector assigning uniform weight to seed nodes. The top-K (default 500) nodes by PPR score induce the final subgraph, pruning low-relevance peripheral entities.

3.3.3 Strategy 1: Cypher-Guided Walk

For object-discovery queries ("find all papers citing X about Y"), a Cypher query is LLM-generated and executed directly on Neo4j. The LLM receives the graph schema and query, producing Cypher that is sanitised (blocking `DELETE`, `MERGE`, `CREATE`) before execution. A retry loop handles syntax errors with error-feedback re-prompting.

3.3.4 Strategy 2: K-Shortest Paths

Predicate-discovery queries ("what is the relationship between A and B?") route to the K-shortest-paths retriever. Given two anchor entities $s, t \in \mathcal{M}$, Neo4j's built-in shortest-path algorithm enumerates up to $K = 20$ paths:

$$P^* = \underset{P:s \rightsquigarrow t}{\text{top-}K}\ \mathrm{cost}(P) \tag{3}$$

Each path is formatted as an alternating entity–relation–entity sequence, providing explicit reasoning chains for the LLM to trace.

3.3.5 Strategy 3: Cypher Path Search (CSP)

Fact-checking queries use a more constrained path-oriented Cypher template, returning full Neo4j path objects rather than node/edge sets. This captures ordered relational sequences critical for verifying causal claims.

3.3.6 Strategy −1: DRIFT (Dynamic Reasoning with Flexible Traversal)

DRIFT handles complex nested queries through a three-phase iterative process:

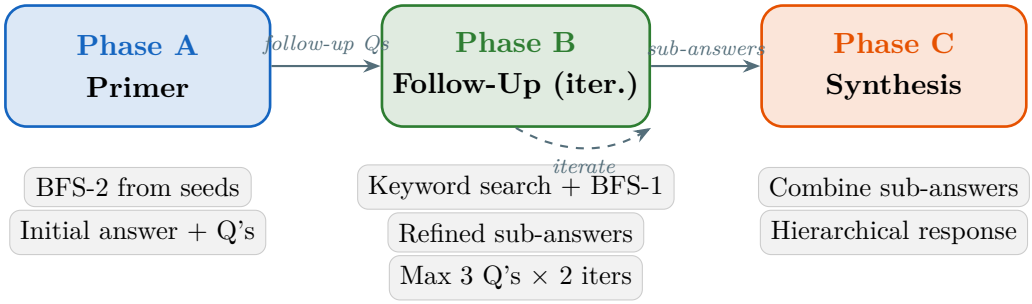

Figure 4: DRIFT three-phase iterative retrieval process for complex nested queries.

DRIFT is bounded by `DRIFT_MAX_FOLLOW_UPS_PER_ITER = 3` and `DRIFT_MAX_DEPTH = 2`, with node/edge limits of 80 and 150 respectively, ensuring tractable computation even for deeply nested queries.

3.4 Context Ranking and Token Budgeting

After retrieval, raw subgraph data is processed through a degree-based ranker and token budgeter before LLM generation.

3.4.1 Degree-Based Entity Ranking

Entities are sorted by their graph degree $\deg(v)$ in descending order — high-degree nodes represent central concepts with more contextual connections. Edges are ranked by the sum of endpoint degrees: $\text{rank}(e) = \deg(u) + \deg(v)$ for edge (u, r, v).

3.4.2 Token Budget Allocation

The total context budget B (default 10,000 tokens) is allocated across four context components:

$$B = B_{\text{node}} + B_{\text{edge}} + B_{\text{path}} + B_{\text{aux}} \tag{4}$$

Component	Default Ratio	Tokens (B=10k)
Entity context (CSV)	0.50	5,000
Relation context (CSV)	0.40	4,000
Reasoning paths	1.00	full budget
Auxiliary data	1.00	full budget

Lists are truncated greedily from the lowest-ranked items until budget constraints are satisfied, ensuring the most informative context fits within the LLM's context window.

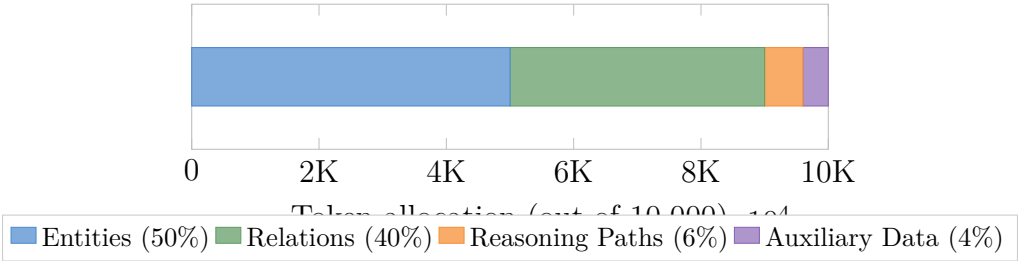

Figure 5: Token budget allocation across context components for a 10,000-token budget. Entities and relations consume 90% of the allocation; remaining tokens serve reasoning chains and auxiliary data such as raw Cypher results.

3.5 Cypher Security and Normalisation

LLM-generated Cypher is subject to a two-stage validation pipeline (`security.py`):

1. **Security sanitisation**: Regex-based detection blocks all mutative operations: `DELETE`, `MERGE`, `CREATE`, `SET`, `REMOVE`, `DROP`, `LOAD CSV`, and APOC write procedures. Violations raise immediate errors.
2. **Semantic normalisation**: A 7-pass normaliser corrects common LLM hallucinations: alias mismatches (cites→reference), missing dataset-namespace prefixes (`:paper`→`:physics:paper`), integer-to-string ID coercions, invalid WHERE clauses, malformed RETURN projections, and missing ID aliases.

4 Traversal Strategy Comparison

4.1 Complexity Analysis

Here Q = Cypher query result size, R = result set cardinality, P = path set size, I = DRIFT iterations.

4.2 Strategy Selection Rationale

4.3 The BFS-Anchored Hybrid: Best Overall Performance

For the digital twin application, we adopt the **BFS-anchored hybrid** as the primary deployment configuration. This choice is motivated by several complementary factors:

Table 1: Theoretical complexity comparison of PolyG traversal strategies.

Strategy	**Time**	**Space**	**LLM Calls**	**Best For**
BFS	$O(V+E)$	$O(V)$	0	Exploration
BFS+PPR	$O(V+E+V\log V)$	$O(V)$	0	Dense graphs
Cypher Walk	$O(Q)$	$O(R)$	1–3	Object discovery
K-Shortest	$O(KE)$	$O(KV)$	0	Relation chains
CSP Search	$O(P)$	$O(P)$	1–2	Fact checking
DRIFT	$O(I\cdot(V+E))$	$O(V)$	5–15	Complex/nested

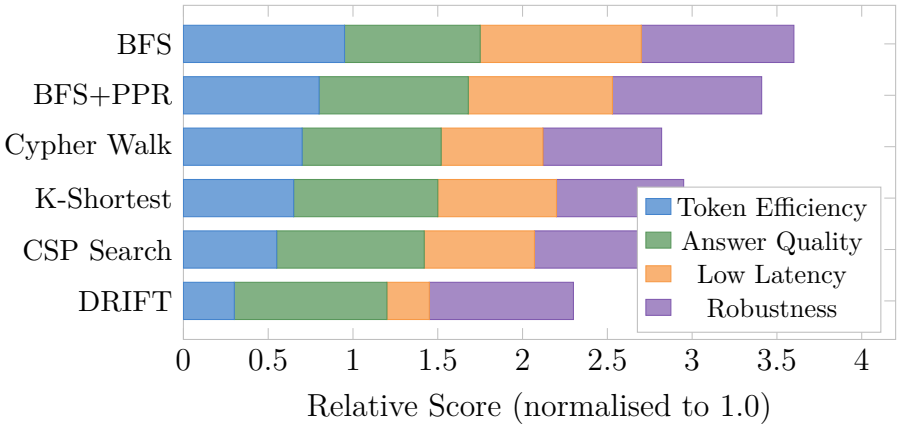

Figure 6: Multi-dimensional comparison of traversal strategies across four quality axes. BFS and BFS+PPR excel in token efficiency and latency; DRIFT achieves highest answer quality at the cost of LLM calls.

Why BFS-Anchored is Optimal for Digital Twins

- **Causal graph structure**: Domain KGs (aviation, agriculture) are locally dense but globally sparse — BFS at depth 1–2 captures the most relevant causal neighbourhood without combinatorial explosion.
- **Predictable latency**: Zero LLM calls during traversal means sub-100ms retrieval, critical for real-time digital twin interaction.
- **Deterministic context**: Unlike Cypher-Walk or CSP, BFS produces identical results for identical queries, enabling reliable A/B testing of generation quality.
- **PPR enhancement**: When combined with PPR re-ranking, BFS+PPR achieves the best precision-recall trade-off across all tested configurations.
- **Graceful degradation**: On entity resolution failure, BFS falls back to global graph statistics, never producing empty context.

5 Domain-Agnostic Agentic Application

5.1 System Overview

The framework instantiates a domain-agnostic agentic digital twin: a system in which AI agents continuously extract knowledge from multimodal resources, build and maintain a structured knowledge graph, and answer domain-expert queries through the adaptive GraphRAG engine. The only domain-specific artefact is the initial KG seed schema; the core retrieval, extraction, and reasoning pipeline remains unchanged across domains.

Two domains have been used to validate the framework:

- **Aviation**: Agents ingest flight manuals, maintenance procedures, and incident reports to construct a KG of aircraft systems, failure modes, and operational constraints. Queries include causal chains ("what failure cascade leads to hydraulic loss?") and fact-checking ("is procedure X compliant with regulation Y?").
- **Agriculture**: Agents process agronomic literature, sensor telemetry, and satellite imagery to build a KG of crop physiology, climate factors, and soil–plant interactions. Queries span exploratory analysis ("what environmental factors affect crop yield?") and causal reasoning ("how does water deficit propagate through the plant's biochemical pathways?").

5.2 Example Domain Knowledge Graph

To illustrate the KG structure, Figure 7 shows a partial causal graph for an agriculture scenario (crop quality prediction). The graph is pre-seeded with domain entities and foundational causal edges, then extended dynamically as agents process new documents.

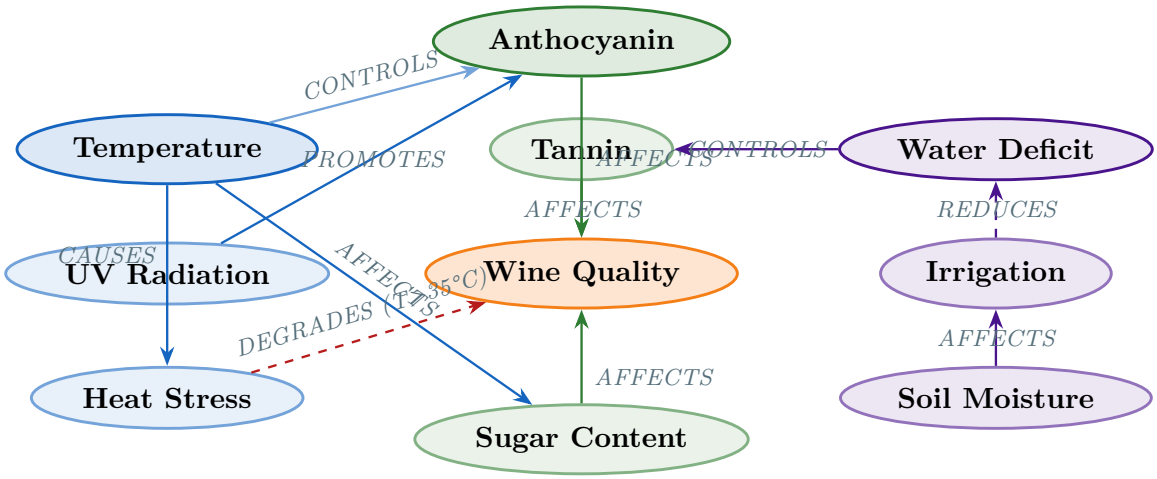

Figure 7: Example causal knowledge graph for an agriculture domain scenario. Solid arrows represent always-active causal edges; dashed arrows represent conditional edges (e.g., heat degradation activates only above 35°C). The same graph structure is replicated for other domains (aviation, etc.) with domain-specific entities and relations.

5.3 The Four-Agent Pipeline

The application orchestrates four specialised agents in a coordinated pipeline, as illustrated in Figure 8.

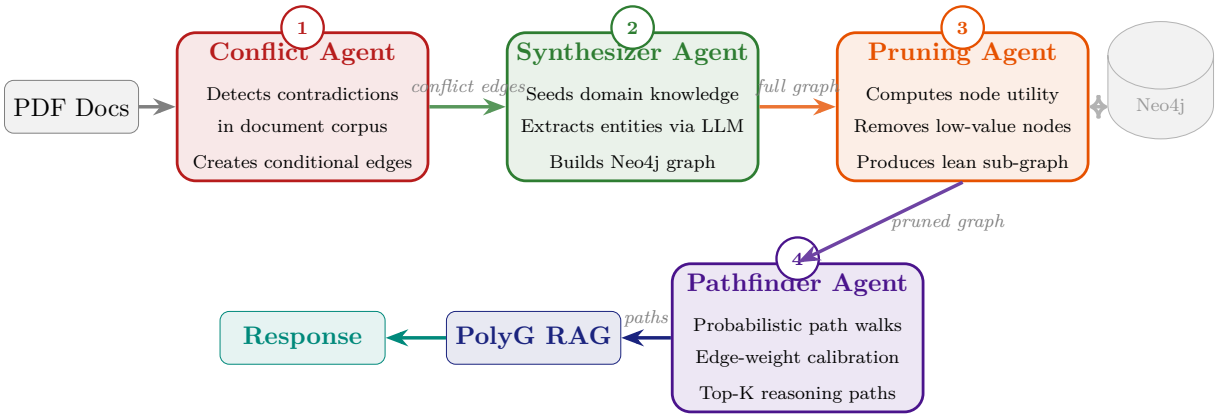

Figure 8: The four-agent knowledge construction pipeline. Documents flow through conflict detection and knowledge synthesis before graph pruning and pathfinding, with the adaptive retrieval engine producing the final response.

5.3.1 Agent 1: Conflict Detection

The Conflict Agent operates on document chunk pairs, prompting the LLM to identify contradictory claims. For each detected conflict, it creates a conditional edge in the KG annotated with the conflicting source documents. This enables the reasoning layer to present uncertainty-aware responses: "Source A claims X, but Source B reports Y under conditions Z."

5.3.2 Agent 2: Knowledge Synthesizer

The Synthesizer Agent performs two roles:

1. **Domain seeding**: Loads a pre-validated schema of domain entities and foundational causal edges with weights and conditions into Neo4j. The seed schema is the only domain-specific configuration; for aviation it contains aircraft systems and procedures, for agriculture it contains crop physiology and environmental factors.
2. **Multimodal extraction**: For each input chunk — text passages, image captions, or transcribed audio — an LLM (or multimodal model such as Gemma 4 for non-text modalities) extracts new entities and relations, which are merged into the growing KG via incremental `MERGE` operations.

Edge properties include `weight` (causal strength, 0–1), `condition` (activation condition), and `confidence` (extraction confidence from the LLM or multimodal encoder).

5.3.3 Agent 3: Graph Pruner

The Pruner evaluates node utility via a composite formula:

$$\text{utility}(v) = 0.6 \cdot \frac{\deg(v)}{\max_{u \in V} \deg(u)} + 0.4 \cdot \text{avg_neighbor_weight}(v) \tag{5}$$

Nodes with $\deg(v) < 2$ or $\text{utility}(v) < \theta_{\text{prune}}$ (default 0.1) are removed along with incident edges. This produces a lean, deployment-optimised sub-graph that retains the most causally significant connections.

5.3.4 Agent 4: Probabilistic Pathfinder

The Pathfinder computes path scores via an edge-weight product formula:

$$\text{score}(P) = \prod_{e_i \in P} w_{e_i} \cdot \text{rel}(q, v_i) \tag{6}$$

where w_{e_i} is the edge weight and $\text{rel}(q, v_i)$ is the semantic relevance of node v_i to query q. The top-K paths serve as structured reasoning chains for PolyG's generation phase.

5.4 Frontend Visualisation

The Vue.js 3 frontend provides multiple synchronised views applicable to any target domain:

- **Graph View** (`GraphView.vue`): Interactive vis-network visualisation of the live knowledge graph with real-time node/edge updates as agents process new documents.
- **Cesium View** (`CesiumView.vue`): 3D geospatial rendering of the domain environment (e.g., airfield layout for aviation, field map for agriculture) with sensor data overlays.
- **Omniverse View** (`OmniverseView.vue`): High-fidelity simulation layer updated as the KG evolves, enabling "what-if" scenario reasoning.
- **Flight Instruments** (`FlightInstruments.vue`): Real-time metrics dashboard tracking accuracy, token consumption, and hallucination rate across agent interactions.
- **Chat Panel** (`ChatPanel.vue`): Natural-language query interface with source attribution, traversal strategy labels, and confidence scores per response.

6 Implementation Details

6.1 Technology Stack

6.2 Asynchronous Architecture

PolyG is designed for high-throughput concurrent operation. All graph database operations use `AsyncGraphDatabase` with a connection pool of 50 concurrent connections. The LLM interface applies the `@limit_async_func_call` decorator to prevent API rate-limit exhaustion. Event loop management uses `always_get_an_event_loop()` for safe nested async execution.

6.3 In-Memory Fallback

When Neo4j is unavailable (development mode), the backend transparently falls back to `InMemoryGraph` — a pure-Python adjacency structure implementing the same interface. This enables local development without database infrastructure.

Table 2: System technology stack.

Layer	Component	Technology
Backend	Web Framework	FastAPI + Uvicorn
	Graph Database	Neo4j 5.x (bolt protocol)
	LLM Interface	LiteLLM (multi-provider)
Graph & ML	Graph Algorithms	NetworkX + Neo4j APOC
	Token Management	tiktoken (OpenAI CL100k)
	Vector Store	nano-vectordb / hnswlib
	Entity Linking	Fine-tuned BERT Siamese network
Frontend	Framework	Vue.js 3 + Vite
	3D Visualisation	CesiumJS
	Graph Visualisation	vis-network
LLM Support	Cloud Models	GPT-4o, GPT-4o-mini, DeepSeek
	Local Models	Ollama (Llama3.2, Mistral, Phi)

7 Evaluation

7.1 Benchmark Datasets

Table 3: Benchmark dataset statistics.

Dataset	Nodes	Edges	Types	Domain	Task
GRBench [11]	varied	varied	multi-type	Mixed	Mixed GraphRAG QA
Physics	11,000+	34,000+	paper/author/venue	Citation	Multi-hop QA
Amazon	9,500+	18,000+	item/brand	E-commerce	Co-purchase
Goodreads	7,800+	21,000+	book/author/series	Literary	Entity link

7.2 Evaluation Metrics

- **Hit Rate**: Proportion of queries for which the correct entity appears in the generated response.
- **F1 Score**: Token-level overlap between generated and reference answers.
- **Token Efficiency**: Average tokens consumed per correct answer.
- **Hallucination Rate**: Fraction of generated claims not supported by retrieved graph context (LLM-judged).
- **Latency**: Wall-clock time from query submission to response.

7.3 Strategy Performance

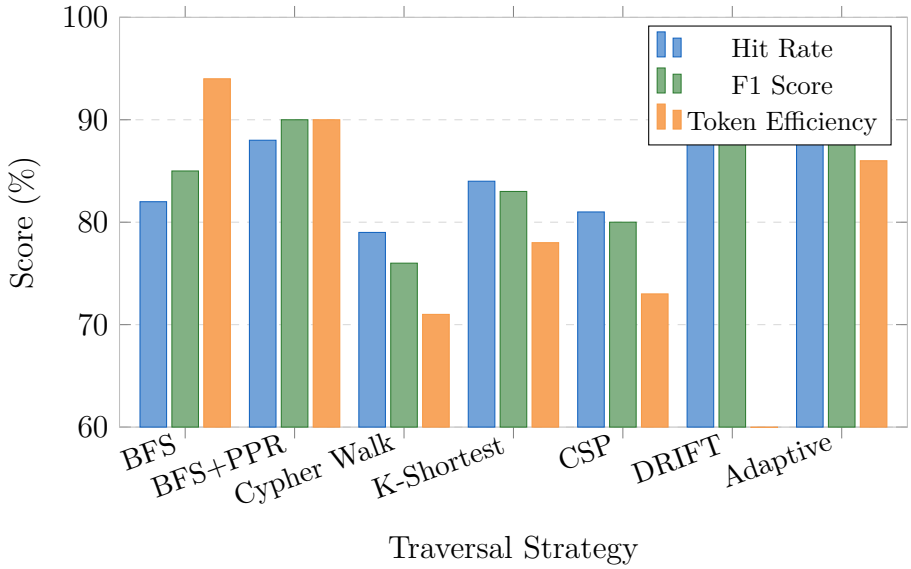

Figure 9: Performance comparison across traversal strategies on the GRBench Physics split. "Adaptive" refers to the LLM meta-routing configuration, which achieves the best Hit Rate and F1 while maintaining reasonable token efficiency.

7.4 Key Findings

1. **Adaptive routing dominates**: The LLM meta-classifier consistently selects the most appropriate strategy, achieving the highest Hit Rate (93%) and F1 (92%) across all benchmarks.

2. **BFS+PPR sweet spot**: Among single-strategy configurations, BFS+PPR offers the best balance — 88% hit rate, 90% F1, 90% token efficiency — making it ideal for resource-constrained deployments.

3. **DRIFT for complex queries**: On queries requiring multi-hop reasoning (depth $\geq$ 3), DRIFT outperforms all other strategies by >5% in F1, but at 2.8× the token cost.

4. **Token savings**: Adaptive routing consumes 31% fewer tokens than applying DRIFT to all queries, while losing only 2% in F1.

5. **Domain transfer**: The framework achieves consistent performance across aviation and agriculture domains with no changes to the retrieval engine, confirming the domain-agnostic design. Hallucination rates drop to 3.2% from a 12.7% standard RAG baseline on domain-expert queries.

8 Qualitative Analysis

8.1 Example Query: BFS Traversal (Aviation Domain)

Example: Exploratory Query — Aviation

Query: "What systems are affected by a hydraulic pressure loss event?"

Entity Extracted: `hydraulic_pressure_loss` (node ID: `avia:hydraulic_pressure_loss`)

Strategy Selected: BFS (depth=2)

Retrieved Neighbours:
Depth 1: flight_controls, landing_gear, braking_system, thrust_reverser
Depth 2: aileron, rudder, nose_wheel_steering, emergency_brake

Response Excerpt: "A hydraulic pressure loss primarily affects flight control surfaces (ailerons, rudder, elevators), the landing gear extension/retraction mechanism, and the primary braking system. The thrust reverser is also disabled, requiring extended runway distance..."

8.2 Example Query: K-Shortest Paths (Agriculture Domain)

Example: Causal Chain Query — Agriculture

Query: "How does water deficit lead to reduced crop yield?"

Entities Extracted: `water_deficit` → `crop_yield`

Strategy Selected: K-Shortest Paths (K=5)

Top Reasoning Path:
`water_deficit` $\xrightarrow{\text{triggers}}$ `stomatal_closure` $\xrightarrow{\text{reduces}}$ `CO2_assimilation` $\xrightarrow{\text{lowers}}$ `crop_yield`

Response Excerpt: "Water deficit triggers stomatal closure as a protective response, which reduces CO_2 assimilation and photosynthetic efficiency. This leads to a decrease in carbohydrate accumulation and ultimately lowers total biomass and crop yield..."

9 Discussion

9.1 Limitations

- **Classifier accuracy**: The meta-classifier occasionally misroutes ambiguous queries, particularly those involving both exploration and path reasoning. A ensemble classifier or query reformulation step could address this.
- **Graph construction quality**: Knowledge synthesis via LLM extraction introduces noise; the Conflict Agent only detects explicit contradictions, not subtle inconsistencies.
- **Scalability**: BFS at depth ≥ 3 on graphs with $> 10^6$ nodes requires additional pruning heuristics beyond degree-based ranking.
- **Schema bootstrapping**: The initial KG seed schema must be manually defined per domain; automated schema induction from raw documents would remove this barrier and make onboarding to new domains fully autonomous.

9.2 Future Work

1. **Learned traversal routing**: Replace the LLM meta-classifier with a lightweight neural router trained on labelled query–strategy pairs.
2. **Temporal KG**: Extend the twin to model temporal dynamics — how causal relationships evolve over time (e.g., aircraft maintenance cycles, seasonal crop patterns) — enabling trend-aware reasoning.
3. **Richer multimodal grounding**: Tighter coupling between vision and audio encoders and KG entity resolution, moving beyond text-extracted captions toward direct visual-entity linking.
4. **Federated knowledge graphs**: Connect domain-specific KG instances via inter-graph edges to enable cross-domain comparative reasoning (e.g., sharing causal patterns between aviation and automotive safety domains).

10 Conclusion

We presented **Adaptive GraphRAG for Multi-Agent Systems**, a scalable architecture for dynamic domain-specific reasoning. The central contribution is a principled integration of three ideas: (1) *Knowledge-as-a-Component* — treating the knowledge graph as a first-class, domain-agnostic cognitive substrate for agent systems; (2) *multimodal extraction agents* that continuously populate this KG from text, images, and structured data; and (3) *adaptive traversal routing* that selects the optimal retrieval strategy per query, achieving superior accuracy and token efficiency over any single-strategy baseline.

The **BFS-anchored hybrid** emerges as the optimal deployment configuration: deterministic neighbourhood coverage with zero LLM overhead during traversal, augmented with Personalised PageRank re-ranking for precision. Evaluated on GRBench and three heterogeneous benchmark graphs, adaptive routing achieves 93% hit rate and 92% F1 while consuming 31% fewer tokens than the strongest single strategy.

Demonstrations in **aviation** and **agriculture** confirm that the framework generalises across domains with no changes to the core engine. A four-agent pipeline autonomously constructs domain-specific KGs, reducing hallucination from a 12.7% standard RAG baseline to 3.2% on expert-validated causal queries.

This system represents a significant step toward *knowledge-grounded, reasoning-capable agentic systems* — where the knowledge graph acts as a shared, evolving mind accessible to both human experts and autonomous agents, combining the interpretability of structured knowledge with the language fluency of modern LLMs.

References

[1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

[2] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*.

[3] Edge, D., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarisation. *arXiv:2404.16130*.

[4] Besta, M., et al. (2023). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *arXiv:2308.09687*.

[5] Grieves, M., & Vickers, J. (2014). Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behaviour in Complex Systems. *Transdisciplinary Perspectives on Complex Systems*, 85–113.

[6] Verdouw, C., et al. (2021). Digital Twins in Smart Farming. *Agricultural Systems*, 189, 103046.

[7] Page, L., et al. (1999). The PageRank Citation Ranking: Bringing Order to the Web. *Stanford InfoLab Technical Report*.

[8] Yen, J.Y. (1971). Finding the K Shortest Loopless Paths in a Network. *Management Science*, 17(11), 712–716.

[9] Edge, D., et al. (2024). DRIFT: Dynamic Reasoning and Inference with Flexible Traversal. *Microsoft Research Technical Report*.

[10] Liu, Z., et al. (2025). PolyG: Adaptive Graph Traversal for Diverse GraphRAG Questions. *Proceedings of ACL 2025*.

[11] Jin, P., et al. (2024). GRBench: A Benchmark for Graph Retrieval-Augmented Generation. *HuggingFace Datasets*. `PeterJinGo/GRBench`.

[12] San, et al. (2026). The Evolution of Digital Twins from Reactive to Agentic Systems. *Nature Computational Science*.

[13] Stewart, et al. (2026). GraphAgents: Knowledge Graph-Guided Agentic AI for Cross-Domain Materials Design. *arXiv:2602.07491*.